

BTVN NHÓM 6

Lê Quang Trung, Nguyễn Hồng Phúc

1 Phát biểu bài toán

Bài toán yêu cầu xử lý một tập hợp n vùng theo dõi hình chữ nhật và m Hunter (điểm) trên bản đồ 2D. Các vùng theo dõi có thể lồng vào nhau nhưng không giao cắt. Nhiệm vụ là tính tổng "Aura" cho mỗi vùng, với quy tắc: mỗi Hunter chỉ được tính vào **vùng theo dõi nhỏ nhất chứa nó**.

2 Chiến lược phát triển thuật toán

Chúng ta sẽ tiếp cận bài toán theo từng bước, tương ứng với các subtask. Mỗi bước sẽ xây dựng dựa trên những hạn chế của bước trước, dẫn đến một giải pháp hoàn chỉnh và tối ưu.

- **Subtask 1:** Giải pháp duyệt đơn giản (Brute-Force).
- **Subtask 2:** Giải pháp tối ưu bằng Sweep-line, áp dụng cho trường hợp tọa độ nhỏ.
- **Subtask 3:** Nâng cấp giải pháp Sweep-line bằng kỹ thuật Rời rạc hóa Tọa độ (Coordinate Compression) để xử lý các tọa độ lớn.

3 Subtask 1: Hướng tiếp cận Brute-Force ($n, m \leq 10^3$)

3.1 Ý tưởng

Với giới hạn n và m nhỏ, ta có thể duyệt qua từng Hunter và kiểm tra với tất cả các vùng theo dõi để tìm ra vùng nhỏ nhất chứa nó.

3.2 Mô tả thuật toán

1. Với mỗi Hunter j , duyệt qua tất cả n vùng theo dõi.
2. Lập danh sách các vùng chứa Hunter j .
3. Nếu danh sách rỗng, cộng Aura vào tổng "không bị theo dõi".
4. Ngược lại, tìm vùng có diện tích nhỏ nhất trong danh sách và cộng Aura vào kết quả của vùng đó.

3.3 Phân tích độ phức tạp

- **Thời gian:** $O(m \times n)$. Với $n, m \leq 10^3$, thuật toán này đủ nhanh.
- **Không gian:** $O(n)$ để lưu kết quả.

3.4 Lý do lựa chọn phương pháp

- **Phù hợp với ràng buộc:** Phương pháp **Brute-Force** có độ phức tạp thời gian là $O(m \times n)$. Với ràng buộc của Subtask 1 là $n, m \leq 10^3$, số lượng phép tính tối đa vào khoảng $10^3 \times 10^3 = 10^6$. Con số này hoàn toàn nằm trong giới hạn cho phép của các hệ thống chấm bài hiện đại (thường khoảng 10^8 phép tính mỗi giây). Do đó, giải pháp này đủ nhanh để vượt qua Subtask 1.
- **Đơn giản và dễ cài đặt:** Đây là phương pháp tiếp cận tự nhiên và thẳng thắn nhất đối với bài toán. Việc cài đặt đơn giản giúp giảm thiểu rủi ro mắc lỗi logic và là một bước khởi đầu tốt để hiểu rõ bản chất bài toán trước khi chuyển sang các thuật toán phức tạp hơn.
- **Nền tảng kiểm thử:** Giải pháp brute-force cung cấp một cơ sở đúng đắn để so sánh và kiểm thử kết quả của các thuật toán tối ưu hơn cho các subtask sau.

4 Sweep-line - Một ứng dụng của Transform and Conquer

Khi giải pháp Brute-Force không còn đáp ứng được yêu cầu về thời gian, chúng ta cần một phương pháp thiết kế thuật toán mạnh mẽ hơn. Bài toán này, vốn thuộc lĩnh vực **Geometric Algorithms**, có thể được giải quyết một cách hiệu quả bằng kỹ thuật Sweep-line.

Phương pháp Sweep-line là một minh chứng kinh điển và xuất sắc cho chiến lược thiết kế **Transform and Conquer**.

4.1 Nguyên lý cốt lõi:

Thay vì xử lý bài toán 2D tính một cách trực tiếp, chúng ta sẽ áp dụng chiến lược Transform and Conquer theo hai giai đoạn rõ rệt:

- **Giai đoạn Transform:** Đây là bước thay đổi cách biểu diễn của bài toán (Representation Change). Chúng ta *biến đổi* các đối tượng hình học 2D tĩnh (hình chữ nhật, điểm) thành một chuỗi các **events** 1D động. Các sự kiện này được sắp xếp trên một trục duy nhất (trục x). Việc biến đổi này giúp giảm chiều của bài toán, làm cho cấu trúc trở nên đơn giản và dễ quản lý hơn. Cụ thể, các sự kiện bao gồm:

– Cạnh trái của một hình chữ nhật.

- Cạnh phải của một hình chữ nhật.
- Vị trí của một Hunter.

Toàn bộ các sự kiện này được đưa vào một **Event Queue**, vốn là kết quả đầu ra của giai đoạn Biến đổi.

- **Giai đoạn Conquer:** Sau khi đã có một biểu diễn mới, ta Conquer bài toán bằng cách xử lý tuần tự các sự kiện trong hàng đợi. Ta tưởng tượng một đường thẳng đứng quét qua mặt phẳng, và tại mỗi sự kiện, đường quét dừng lại để ta giải quyết một bài toán con 1D đơn giản hơn. Để làm được điều này, ta cần một **Sweep-Line Status** – một cấu trúc dữ liệu lưu trữ thông tin về các đối tượng đang giao với đường quét tại vị trí hiện tại. Việc cập nhật và truy vấn trên cấu trúc dữ liệu này chính là quá trình conquer bài toán.

4.2 Các thành phần chính trong ứng dụng

Việc áp dụng mô hình trên vào bài toán "Aura Tracker" được cụ thể hóa như sau:

1. **Hàng đợi sự kiện:** Một danh sách chứa tất cả các sự kiện (OPEN, CLOSE, QUERY), được sắp xếp theo tọa độ x . Đây là sản phẩm của giai đoạn **Transform**.
2. **Cấu trúc dữ liệu Trạng thái:** Đây là "trái tim" của giai đoạn **Conquer**. Nó cần phải lưu trữ hiệu quả tập hợp các vùng đang "kích hoạt" (active) và hỗ trợ các thao tác:
 - Thêm một vùng (khi gặp sự kiện OPEN).
 - Xóa một vùng (khi gặp sự kiện CLOSE).
 - Truy vấn để tìm vùng nhỏ nhất chứa một điểm y cho trước (khi gặp sự kiện QUERY).

Trong các phần tiếp theo, chúng ta sẽ thấy rằng sự khác biệt trong việc giải quyết Subtask 2 và Subtask 3 chính là ở cách chúng ta lựa chọn và cài đặt Cấu trúc dữ liệu Trạng thái này để phù hợp với từng loại ràng buộc.

5 Subtask 2: Áp dụng Sweep-line cho tọa độ nhỏ

5.1 Ý tưởng

Khi n, m lớn, thuật toán $O(m \times n)$ sẽ quá chậm. Ta chuyển sang thuật toán Sweep-line. Ta sẽ quét một đường thẳng đứng từ trái sang phải, xử lý các sự kiện tại các tọa độ x quan trọng.

5.2 Thiết kế chi tiết

1. Định nghĩa sự kiện:

- **OPEN (type=0):** Cạnh trái của hình chữ nhật ($x = x_1$).
- **QUERY (type=1):** Vị trí của Hunter ($x = C_j$).
- **CLOSE (type=2):** Cạnh phải của hình chữ nhật ($x = x_2$).

2. Sắp xếp sự kiện: Gộp tất cả sự kiện và sắp xếp theo x , nếu x bằng nhau thì ưu tiên $\text{OPEN} < \text{QUERY} < \text{CLOSE}$.

3. CTDL Active Set: Ta cần một CTDL để lưu các vùng đang bị đường quét cắt qua. Vì tọa độ y nhỏ, ta có thể dùng **Segment Tree** trên dải tọa độ y từ 0 đến H .

- Mỗi nút của cây sẽ lưu (**area**, **idx**) của vùng có diện tích nhỏ nhất đang bao phủ hoàn toàn đoạn y tương ứng.
- Ta dùng kỹ thuật Lazy Update để cập nhật giá trị mỗi nút.

4. Xử lý sự kiện:

- **Gặp sự kiện OPEN của vùng i :** Cập nhật đoạn $[y_1, y_2 - 1]$ trên Segment Tree với thông tin (**area_i**, **idx_i**). Phép cập nhật là **min** theo diện tích.
- **Gặp sự kiện CLOSE của vùng i :** "Xóa" ảnh hưởng của vùng i bằng cách cập nhật lại đoạn $[y_1, y_2 - 1]$ với giá trị trước đó. Điều này đòi hỏi phải xây dựng lại trạng thái, khá phức tạp nhưng khả thi.
- **Gặp sự kiện QUERY tại điểm (x, y) :** Truy vấn điểm tại vị trí y trên Segment Tree để lấy (**area**, **idx**) của vùng nhỏ nhất chứa điểm đó.

5.3 Phân tích độ phức tạp

- **Thời gian:** $O((n + m) \log(n + m) + (n + m) \log H)$. Bước sắp xếp sự kiện là $O((n + m) \log(n + m))$, và mỗi thao tác trên Segment Tree là $O(\log H)$.
- **Không gian:** $O(n + m + H)$ để lưu sự kiện và Segment Tree.

5.4 Lý do lựa chọn phương pháp

- **Vượt qua giới hạn của Brute-Force:** Với n, m có thể lên tới 10^5 , thuật toán $O(m \times n)$ sẽ cần tới 10^{10} phép tính, không còn khả thi. Do đó, ta cần một thuật toán hiệu quả hơn.
- **Tận dụng ràng buộc tọa độ nhỏ:** Điểm mấu chốt của Subtask 2 là ràng buộc $W, H \leq 10^3$. Ràng buộc này gợi ý rằng độ phức tạp của thuật toán nên phụ thuộc vào kích thước của không gian tọa độ (H) thay vì số lượng đối tượng (n). Phương pháp **Sweep-line kết hợp Segment Tree** là lựa chọn lý tưởng cho kịch bản này.

- **Hiệu quả của cấu trúc dữ liệu:** Thuật toán Sweep-line giúp giảm chiều của bài toán từ 2D xuống 1D. Tại mỗi bước quét, ta cần xử lý một bài toán trên trục y . Segment Tree được xây dựng trên dải tọa độ y (kích thước H) cho phép thực hiện các thao tác cập nhật đoạn và truy vấn điểm trong thời gian $O(\log H)$. Vì H nhỏ, các thao tác này cực kỳ nhanh chóng. Sự kết hợp này giúp giải quyết bài toán hiệu quả mà không bị ảnh hưởng bởi số lượng lớn các đối tượng n, m .

6 Subtask 3: Sweep-line và Rời rạc hóa Tọa độ

6.1 Vấn đề với tọa độ lớn

Khi tọa độ Y có thể lên đến 10^9 , việc xây dựng một Segment Tree có kích thước 10^9 là bất khả thi về cả thời gian và bộ nhớ.

6.2 Kỹ thuật Rời rạc hóa Tọa độ (Coordinate Compression)

Đây là kỹ thuật mấu chốt để giải quyết vấn đề.

1. **Thu thập:** Tập hợp tất cả các tọa độ y quan trọng vào một mảng: các giá trị y_1, y_2 từ các hình chữ nhật và y từ các Hunter.
2. **Lọc và sắp xếp:** Loại bỏ các giá trị trùng lặp và sắp xếp mảng các tọa độ y duy nhất này.
3. **Ánh xạ:** Tạo một bản đồ (map) để ánh xạ mỗi tọa độ y gốc sang một chỉ số mới, nhỏ gọn (ví dụ từ 0 đến $K - 1$, với K là số lượng tọa độ y duy nhất). K sẽ có kích thước tối đa là $2n + m$.

6.3 Áp dụng vào thuật toán Sweep-line

Sau khi rời rạc hóa, ta áp dụng lại thuật toán Sweep-line, nhưng thay vì xây dựng Segment Tree trên dải tọa độ $[0, H]$, ta sẽ xây dựng nó trên dải chỉ số mới đã được rời rạc hóa $[0, K - 1]$.

- Mọi tọa độ y trong các sự kiện (OPEN, CLOSE, QUERY) đều được thay thế bằng chỉ số tương ứng sau khi rời rạc hóa.
- Segment Tree bây giờ chỉ cần kích thước $O(K) = O(n + m)$.

Cài đặt trong thực tế (như mã nguồn cung cấp) có thể dùng một cấu trúc dữ liệu khác thay cho Segment Tree. Một danh sách được sắp xếp (sorted list) kết hợp với tìm kiếm nhị phân (bisect trong Python) có thể mô phỏng lại logic tìm kiếm vùng nhỏ nhất một cách hiệu quả mà không cần rời rạc hóa một cách tường minh, vì nó hoạt động trực tiếp trên các giá trị y .

6.4 Phân tích độ phức tạp

- **Thời gian:**

- Rời rạc hóa: $O((n+m)\log(n+m))$.
- Sắp xếp sự kiện: $O((n+m)\log(n+m))$.
- Xử lý sự kiện với Segment Tree trên K phần tử: $O((n+m)\log K) = O((n+m)\log(n+m))$.
- Tổng thời gian: $O((n+m)\log(n+m))$.

- **Không gian:** $O(n+m)$ cho sự kiện, bản đồ ánh xạ, và Segment Tree.

6.5 Lý do lựa chọn phương pháp

- **Vấn đề của giải pháp trước:** Giải pháp cho Subtask 2 sử dụng Segment Tree trên toàn bộ dải tọa độ y (từ 0 đến H). Khi H có thể lên tới 10^9 , việc xây dựng một Segment Tree có kích thước lớn như vậy là bất khả thi về cả bộ nhớ và thời gian.
- **Bản chất thừa thớt của dữ liệu:** Mặc dù dải tọa độ rất lớn, số lượng tọa độ y thực sự quan trọng (các cạnh trên/dưới của hình chữ nhật và vị trí của Hunter) chỉ là $O(n+m)$. Điều này có nghĩa là dữ liệu của chúng ta "thừa thớt". Kỹ thuật **Rời rạc hóa Tọa độ (Coordinate Compression)** được sinh ra để giải quyết chính xác vấn đề này.
- **Loại bỏ sự phụ thuộc vào giá trị tọa độ:** Rời rạc hóa ánh xạ các tọa độ lớn và thừa thớt sang một tập hợp các chỉ số nhỏ và liên kề (từ 0 đến $K-1$, với $K \approx 2n+m$). Kỹ thuật này giữ nguyên quan hệ thứ tự tương đối giữa các tọa độ nhưng loại bỏ hoàn toàn sự phụ thuộc vào giá trị tuyệt đối của chúng. Sau khi rời rạc hóa, ta có thể áp dụng lại thuật toán Sweep-line và Segment Tree trên không gian mới có kích thước chỉ $O(n+m)$. Độ phức tạp của mỗi thao tác trên cây lúc này là $O(\log K) = O(\log(n+m))$, hoàn toàn độc lập với H . Đây là phương pháp tiêu chuẩn và hiệu quả nhất để giải quyết bài toán trong trường hợp tổng quát.

7 Mã nguồn cài đặt tham khảo

```
1 import sys
2 from bisect import bisect_right, bisect_left, insort
3
4 def main():
5     # Đọc input nhanh từ buffer
6     data = list(map(int, sys.stdin.buffer.read().split()))
7     it = iter(data)
```

```

8     W, H, n, m = next(it), next(it), next(it), next(it)
9
10    rects = [(next(it), next(it), next(it), next(it), i) for i in range(n)]
11    points = [(next(it), next(it), next(it)) for _ in range(m)]
12
13    results = [0] * n
14    untracked_aura = 0
15
16    events = []
17    # (x, type, v1, v2, idx), type: 0=OPEN, 1=QUERY, 2=CLOSE
18    for x1, y1, x2, y2, idx in rects:
19        events.append((x1, 0, y1, y2, idx))
20        events.append((x2, 2, y1, y2, idx))
21
22    for x, y, c in points:
23        events.append((x, 1, y, c, -1))
24
25    events.sort()
26
27    # Active Set: lưu các (y1, y2, idx) đang active
28    active = []
29    for x, typ, v1, v2, idx in events:
30        if typ == 0: # OPEN
31            insort(active, (v1, v2, idx))
32        elif typ == 2: # CLOSE
33            pos = bisect_left(active, (v1, v2, idx))
34            if pos < len(active) and active[pos] == (v1, v2, idx):
35                active.pop(pos)
36        else: # QUERY
37            y, c = v1, v2
38            # Tìm ứng viên: hình chữ nhật có y1 lớn nhất mà y1 < y
39            pos = bisect_right(active, (y, -1, -1)) - 1
40            found = False
41            if pos >= 0:
42                y1_cand, y2_cand, idx_cand = active[pos]
43                if y1_cand < y < y2_cand:
44                    results[idx_cand] += c
45                    found = True
46            if not found:
47                untracked_aura += c
48
49    final_results = results + [untracked_aura]

```

```
50     print(" ".join(map(str, final_results)))
51
52 if __name__ == "__main__":
53     main()
```
